Laboratory 8 – Exercises

1. Apply the linear regression model on the **Wine Quality (red)** dataset, which contains 1599 examples, each consisting of 11 numerical features describing various properties of red wines, together with a target quality score (an integer between 3 and 8). Standardize all input features by subtracting the mean and dividing by the standard deviation of each feature (computed on the training set only). Split the dataset into a training set (80%) and a testing set (20%).

   To load the dataset, you may use the following code snippet:

   ```
   from sklearn.datasets import fetch_openml

   data = fetch_openml(name="wine-quality-red", version=1, as_frame=True)
   df = data.frame

   X = df.drop("class", axis=1).values
   y = df["class"].astype(np.float32).values
   ```

   Create the linear regression model (**do not use PyTorch's predefined layers**) and train it as follows:

   - initialize all weights and the bias term to 0;
   - set the batch size to 32;
   - implement the mean squared error (MSE) loss;
   - implement the mini-batch gradient descent optimizer with learning rate $lr = 0.01$ (**do not** use the `torch.optim` package);
   - set the total number of epochs to 40;
   - train the model on the training set and record the training loss after each epoch.

   After training is completed:

   - plot the loss curve across all epochs;
   - evaluate the model on the test set and compute the Mean Squared Error (MSE) and Mean Absolute Error (MAE).

2. Use the **Oxford-IIIT Pet** dataset from `torchvision.datasets`, which contains 7,349 images of different cat and dog breeds. Each image has a corresponding class label in the range $[0, 36]$. The dataset is split into two official subsets: *trainval* (training + validation data) and *test*.

   Preprocess all images as follows:

   - resize each image to a fixed resolution of 64×64 pixels;

- convert each image to a PyTorch tensor with pixel values in the interval $[0, 1]$.

Use the entire *trainval* split to build a training set and a validation set by performing an 80% / 20% random split. Use the separate official *test* split as the final test set.

Build a multilayer perceptron (MLP) classifier using PyTorch, following these rules:

- 2 hidden layers (first layer consists of 512 hidden units and the second layer 256 hidden units) with the rectified linear unit activation function;
- a dropout function after the first hidden layer (dropout probability 10%.

Train the model under the following settings:

- use the Cross-Entropy Loss;
- use the Adam optimizer with learning rate $lr = 0.001$;
- set the batch size to 64;
- train the model for 15 epochs;
- after each epoch, compute and record:
  - training loss;
  - training accuracy;
  - validation loss;
  - validation accuracy.

After training is completed:

- plot both the training and validation loss curves;
- plot the training and validation accuracy curves;
- compute the test accuracy.

3. Generate an artificial regression dataset with 3 input features and a nonlinear target function. Each sample $x = (x_0, x_1, x_2)$ is drawn independently from the uniform distribution on $[-2, 2]^3$. The target value is defined as:

$$y = \sin(x_0) + 0.5 \cdot x_1^2 - x_2 + \varepsilon,$$

where $\varepsilon \sim \mathcal{N}(0, 0.1^2)$ is Gaussian noise.

- generate 10,000 samples and split them into a training set (70%), a validation set (15%), and a test set (15%);
- build an MLP regression model in PyTorch , the MLP architecture will consist of 3 hidden layer with of 64 hidden units with rectified linear unit activation function;
- train the model with:
  - loss: Mean Squared Error (MSE);

  - optimizer: Adam with learning rate $lr = 0.001$;
  - batch size: 64;
  - number of epochs: 50.

After training:

- plot the training and validation loss curves;
- compute the MSE and MAE on the test set.